

SplitStack Project Charter

Version: 1.0

Date: May 12, 2026

Project: SplitStack shared budgeting and expense management app

Purpose

SplitStack helps roommates, travel groups, families, and other shared-cost groups track expenses, split balances fairly, approve large purchases, record settlements, and understand spending patterns from one workspace.

Business Need

Shared expenses are often tracked through informal chats, spreadsheets, payment apps, and screenshots. This creates missed reimbursements, unclear balances, and disagreements over larger purchases. SplitStack provides one system for group membership, expense entry, voting, settlements, budget tracking, analytics, receipt capture, and AI-assisted financial guidance.

Project Objectives

- Provide account registration and login for individual users.
- Let users create shared groups, invite members, and manage accepted memberships.
- Let group members add expenses with equal, percent, and custom split methods.
- Require group approval for expenses above a configurable threshold.
- Calculate balances and outstanding payment relationships from approved expenses.
- Let users mark expenses paid and notify the payer/payee.
- Support receipt upload and OCR-assisted expense entry.
- Provide personal budget tracking and spending analytics.
- Provide a SplitStack assistant that answers questions using workspace data.
- Provide a web client, mobile client, backend API, persistent database, and project documentation.

Scope

In scope:

- React/Vite web app.
- Flutter mobile app.
- Express API.
- SQLite persistence.
- Local/demo deployment instructions.
- Functional, technical, and QA documentation.

Out of scope for this submission:

- Production cloud deployment.
- Real payment transfer processing.
- Enterprise identity provider integration.
- Public app-store distribution.
- Production-grade observability and backup operations.

Stakeholders

Role	Responsibility
Project Manager	Project coordination, scope, schedule, and final submission organization.
Business Analyst	Requirements, use cases, activity flows, and functional acceptance criteria.
Developers	Web, mobile, backend, database, integrations, and technical documentation.
QA	Test strategy, test cases, traceability, and test results summary.
End Users	Group members and group owners who manage shared spending.
Instructor	Reviews deliverables and evaluates project completeness.

Major Deliverables

- Project charter.
- Functional and requirements specification.
- Use cases.
- Activity diagrams.
- Domain object model or ER diagram.
- Technical/design specification.
- Context/deployment diagram.
- Architecture layout.
- Component diagram.
- Class hierarchy and relationship diagrams.
- Sequence diagrams.
- Test strategy, test plan, traceability matrix, test cases, and test results summary.
- Source code.
- Build and deployment instructions.
- Release notes.

Assumptions

- Users have access to a browser or Flutter-supported device/emulator.

- The backend runs locally for the final project demo.
- SQLite is sufficient for class-project persistence and demo workflows.
- Gemini assistant behavior is optional and falls back to local summaries if no API key is configured.
- Real payment transfer is represented by recorded settlements, not by transferring money.

Constraints

- The project uses Node.js 22 LTS for the website/backend.
- The mobile client depends on Flutter SDK availability.
- Local mobile simulator setup depends on Xcode for iOS and Android Studio for Android.
- The final release is not tagged in git; it is based on the current `main` branch.

Risks

Risk	Impact	Mitigation
Local environment differences	Demo or build issues	Provide separate web, backend, and mobile setup instructions.
Sparse sample data	AI and analytics appear limited	Use real registered demo accounts and add several test expenses before demo.
Flutter SDK missing on reviewer machine	Mobile app cannot be run locally	Include web client as primary runnable client and document Flutter prerequisites.
AI API key unavailable	Assistant cannot call Gemini	Backend local fallback answers workspace questions without an API key.
SQLite is local-only	Not production ready	State local/demo scope clearly in release notes and known limitations.

Success Criteria

- A user can register, log in, create or join a group, add expenses, and view updated balances.
- Above-threshold expenses create votes and only affect balances after approval.
- Users can record settlements and see payment status update.
- Users can create challenges and add contributions.
- Users can set budgets and review spending analytics.
- The assistant can answer workspace questions or provide fallback summaries.
- The source code builds and the documentation maps to every requested deliverable.

Approval

This charter defines the intended final submission scope for SplitStack 1.0.0.